hw3

November 25, 2025

1 Homework 3: Supervised machine learning

UIC CS 418, Fall 2025

*According to the **Academic Integrity Policy** of this course, all work submitted for grading must be done individually, unless otherwise specified. While we encourage you to talk to your peers and learn from them, this interaction must be superficial with regards to all work submitted for grading. This means you cannot work in teams, you cannot work side-by-side, you cannot submit someone else's work (partial or complete) as your own. In particular, note that you are guilty of academic dishonesty if you extend or receive any kind of unauthorized assistance. Absolutely no transfer of program code between students is permitted (paper or electronic), and you may not solicit code from family, friends, or online forums. Other examples of academic dishonesty include emailing your program to another student, copying-pasting code from the internet, working in a group on a homework assignment, and allowing a tutor, TA, or another individual to write an answer for you. Academic dishonesty is unacceptable, and penalties range from failure to expulsion from the university; cases are handled via the official student conduct process described at https://dos.uic.edu/conductforstudents.shtml.*

This homework is an individual assignment for all graduate students. Undergraduate students are allowed to work in pairs and submit one homework assignment per pair. There will be no extra credit given to undergraduate students who choose to work alone. The pairs of students who choose to work together and submit one homework assignment together still need to abide by the Academic Integrity Policy and not share or receive help from others (except each other).

1.1 Due Date

This assignment is due at 11:59pm Wednesday, November 26th, 2025.

1.1.1 What to Submit

You need to complete all code and answer all questions denoted by **Q#** in this notebook. When you are done, you should export `hw3.ipynb` with your answers as a PDF file, upload that file `hw3.pdf` to *Homework 3 - Written Part* on Gradescope, tagging each question.

You need to copy all functions that are part of questions Q1-Q9 to `hw3.py`. That includes `process()`, `process_all()`, `create_features()`, `create_labels()`, `class MajorityLabelClassifier()`, `learn_classifier()`, `evaluate_classifier()`, `best_model_selection()` and `classify_tweets()`. You need to upload a completed Jupyter notebook (`hw3.ipynb` file) and `hw3.py` to *Homework 3 - code* on Gradescope. To help you get

started, we have provided a template file (`hw3_template.py`) containing imports, and function skeletons.

For undergraduate students who work in a team of two, only one student needs to submit the homework and just tag the other student on Gradescope. As always, be sure to mark the full problems as specified in Gradescope.

Autograding Questions will be graded based on both manual grading and an Autograder which will run on your `hw3.py` file. This assignment is graded on the basis of correctness and 70/100 points are given by the autograder. The remaining 30 points will be manually graded (10 points for Q8, 10 points for Q9 and 10 points for correctly running everything in the Jupyter notebook).

The test cases will take a bit longer to execute. Make use of the resources wisely by first testing your functions in your notebook or making local test cases.

[41]:
```
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
import nltk #install using "conda install anaconda::nltk"
import sklearn
import string
import re # helps you filter urls
from IPython.display import display, Latex, Markdown
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

2 Classifying tweets

In this problem, you will be analyzing Twitter data extracted using the Twitter API. We have provided the data containing tweets posted by the following three X/Twitter accounts: `ZohranKMamdani, CurtisSliwa, andrewcuomo` in `train.csv` file.

The csv file contains the following columns: - `Content`: type of content - `handle` : name of the X account

The tweets have been divided into two parts - train and test available to you in CSV files. For train, both the `handle` and `Content` attributes were provided but for test, `handle` is hidden.

The overarching goal of the problem is to "predict" the political inclination or affiliation (Democratic/Republican/Independent) of individual tweets. The ground truth (i.e., true class labels) is determined from the `handle` of the tweet as follows - `Zohran Mamdani` is Democrat - `Curtis Sliwa` is Republican - `Andrew Cuomo` is Independent

Thus, this is a 3-class classification problem.

We will The problem proceeds in three stages: - **Text processing (15%)**: We will clean up the raw tweet text using the various functions offered by the nltk package. - **Feature construction (25%)**: In this part, we will construct bag-of-words feature vectors and training labels from the

processed text of tweets and the `screen_name` columns respectively. - **Classification (60%)**: Using the features derived, we will use sklearn package to learn a model which classifies the tweets as desired.

You will use two new python packages in this problem: `nltk` and `sklearn`, both of which should be available with anaconda. However, NLTK comes with many corpora, toy grammars, trained models, etc, which have to be downloaded manually. This assignment requires NLTK's stopwords list, POS tagger, and WordNetLemmatizer. Install them using conda

[11]:
```
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt')    # Use nltk downloader to download resource "punkt" once
nltk.download('punkt_tab')    # Use nltk downloader to download resource "punkt_tab" once

from nltk import pos_tag
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords

# Verify that the following commands work for you, before moving on.

lemmatizer= nltk.stem.wordnet.WordNetLemmatizer()
stopwords= stopwords.words('english')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Dylan\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Dylan\AppData\Roaming\nltk_data…
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\Dylan\AppData\Roaming\nltk_data…
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Dylan\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Dylan\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
```

[12]:
```
train = pd.read_csv("train.csv")
train_tweets = train["Content"]
print(train_tweets.head(),'\n',len(train_tweets))
```

```
0    This is a disgusting and heartbreaking act of …
1    Honored to join Richmond Hill's Hindu communit…
2    New York, tomorrow is the big day, the final p…
```

```
3    The Art of the Deal.Breaking News: Mayor Eric …
4    October 7th was a dark day, and the memory of …
Name: Content, dtype: object
 913
```

Let's begin!

2.1 A. Text Processing [15%]

You first task to fill in the following function which processes and tokenizes raw text. The generated list of tokens should meet the following specifications: 1. The tokens must all be in lower case. 2. The tokens should appear in the same order as in the raw text. 3. The tokens must be in their lemmatized form. If a word cannot be lemmatized (i.e, you get an exception), simply catch it and ignore it. These words will not appear in the token list. 4. The tokens must not contain any punctuations. Punctuations should be handled as follows: (a) Apostrophe of the form `'s` must be ignored. e.g., `She's` becomes `she`. (b) Other apostrophes should be omitted. e.g, `don't` becomes `dont`. (c) Words must be broken at the hyphen and other punctuations. 5. The tokens must not contain any part of a url.

Part of your work is to figure out a logical order to carry out the above operations. You may find `string.punctuation` useful, to get hold of all punctuation symbols. Look for regular expressions capturing urls in the text. Your tokens must be of type `str`. Use `nltk.word_tokenize()` for tokenization once you have handled punctuation in the manner specified above.

You would want to take a look at the `lemmatize()` function here.

In order for `lemmatize()` to give you the root form for any word, you have to provide the context in which you want to lemmatize through the `pos` parameter: `lemmatizer.lemmatize(word, pos=SOMEVALUE)`. The context should be the part of speech (POS) for that word. The good news is you do not have to manually write out the lexical categories for each word because nltk.pos_tag() will do this for you. Now you just need to use the results from `pos_tag()` for the `pos` parameter. However, you can notice the POS tag returned from `pos_tag()` is in different format than the expected pos by `lemmatizer`.

pos

(Syntactic category): n for noun files, v for verb files, a for adjective files, r for adverb words.

You need to map these pos appropriately. `nltk.help.upenn_tagset()` provides description of each tag returned by `pos_tag()`.

2.2 Q1 (6%):

[13]:
```
# As you can see, some rows are Nans (perhaps it was just a video/image post␣
↪with no text), so we have to remove them.
#
# insert code to remove nan rows
train_tweets = train_tweets.dropna()
print(train_tweets.isna().sum())
```

```
0
```

[14]:
```
# now we will look at tweet 47 to get an idea of other symbols, emojis that need to be removed

print(train_tweets[47])

#basic tokenization
tokens = word_tokenize(train_tweets[47])
print(tokens,'\n')

# we do not want emojis, so this will do it -- read the documentation related to it here: https://docs.python.org/3/library/re.html#re.compile
emoji_pattern = re.compile("["
        u"\U0001F600-\U0001F64F"  # emoticons
        u"\U0001F300-\U0001F5FF"  # symbols & pictographs
        u"\U0001F680-\U0001F6FF"  # transport & map symbols
        u"\U0001F1E0-\U0001F1FF"  # flags (iOS)
        u"\U00002702-\U000027B0"  # dingbats
        u"\U000024C2-\U0001F251"  # misc
                           "]+", flags=re.UNICODE)


print(emoji_pattern.sub(r'', train_tweets[47])) # no emoji
tokens = word_tokenize(emoji_pattern.sub(r'', train_tweets[47]))
print('No emojis:','\n',tokens,'\n') #no emojis

#similarly have to handle urls, and other requirements.
```

```
This Sunday, we’re SHREDDING

Join me (or someone who looks a lot like me) at one of two locations to safely
dispose of old bills, bank statements and documents you no longer need.

+ DJs, family portraits and frozen treats.

12-3pm in Harlem and the Bx. Details in the vid
['This', 'Sunday', ',', 'we', '’', 're', 'SHREDDING', ' ', 'Join', 'me', '(',
'or', 'someone', 'who', 'looks', 'a', 'lot', 'like', 'me', ')', 'at', 'one',
'of', 'two', 'locations', 'to', 'safely', 'dispose', 'of', 'old', 'bills', ',',
'bank', 'statements', 'and', 'documents', 'you', 'no', 'longer', 'need', '.',
'+', 'DJs', ',', 'family', 'portraits', 'and', 'frozen', 'treats', '.',
'12-3pm', 'in', 'Harlem', 'and', 'the', 'Bx', '.', 'Details', 'in', 'the',
'vid', ' ']

This Sunday, we’re SHREDDING

Join me (or someone who looks a lot like me) at one of two locations to safely
dispose of old bills, bank statements and documents you no longer need.
```

```
+ DJs, family portraits and frozen treats.

12-3pm in Harlem and the Bx. Details in the vid
No emojis:
 ['This', 'Sunday', ',', 'we', '’', 're', 'SHREDDING', 'Join', 'me', '(', 'or',
'someone', 'who', 'looks', 'a', 'lot', 'like', 'me', ')', 'at', 'one', 'of',
'two', 'locations', 'to', 'safely', 'dispose', 'of', 'old', 'bills', ',',
'bank', 'statements', 'and', 'documents', 'you', 'no', 'longer', 'need', '.',
'+', 'DJs', ',', 'family', 'portraits', 'and', 'frozen', 'treats', '.',
'12-3pm', 'in', 'Harlem', 'and', 'the', 'Bx', '.', 'Details', 'in', 'the',
'vid']
```

[15]:
```
#tweet processing example
example_tweet = "  making world's best is what we're proud of doing␣
 ↪off-the-shelf! And more proud:https://www.google.com"
#tempstr = gop_tweets[temp_num]
print(example_tweet)

emoji_pattern = re.compile("["
        u"\U0001F600-\U0001F64F"  # emoticons
        u"\U0001F300-\U0001F5FF"  # symbols & pictographs
        u"\U0001F680-\U0001F6FF"  # transport & map symbols
        u"\U0001F1E0-\U0001F1FF"  # flags (iOS)
        u"\U00002702-\U000027B0"  # dingbats
        u"\U000024C2-\U0001F251"  # misc
                           "]+", flags=re.UNICODE)

#handle url
print(re.sub(r'http\S+', '', example_tweet))
example_tweet = re.sub(r'http\S+', '', example_tweet)

#handle emojis
print(emoji_pattern.sub(r'', example_tweet)) # no emoji
example_tweet = emoji_pattern.sub(r'', example_tweet)

#handle apostrophe 's
print(example_tweet)
print(example_tweet.replace("'s","")) #replace 's to
example_tweet = example_tweet.replace("'s","")

#handle other apostrophe
example_tweet = example_tweet.replace("'","")

#handle all other punctuation
print(re.sub(r'[' + string.punctuation + r']+', ' ', example_tweet).strip())
```

```
example_tweet = re.sub(r'[' + string.punctuation + r']+', ' ', example_tweet).
 ↪strip()

#handle lowercase
example_tweet = example_tweet.lower()

tokens = word_tokenize(example_tweet)

print(tokens,'\n')

#lemmatize single word or token
token_lemma = lemmatizer.lemmatize(tokens[0])
print(token_lemma)

#lemmatize a list of workds or tokens
tweet_lemma = [lemmatizer.lemmatize(token) for token in tokens]
print(tweet_lemma)
```

```
  making world's best is what we're proud of doing off-the-shelf! And more
proud:https://www.google.com
  making world's best is what we're proud of doing off-the-shelf! And more
proud:
 making world's best is what we're proud of doing off-the-shelf! And more proud:
 making world's best is what we're proud of doing off-the-shelf! And more proud:
 making world best is what we're proud of doing off-the-shelf! And more proud:
making world best is what were proud of doing off the shelf  And more proud
['making', 'world', 'best', 'is', 'what', 'were', 'proud', 'of', 'doing', 'off',
'the', 'shelf', 'and', 'more', 'proud']

making
['making', 'world', 'best', 'is', 'what', 'were', 'proud', 'of', 'doing', 'off',
'the', 'shelf', 'and', 'more', 'proud']
```

[16]:
```
# Convert part of speech tag from nltk.pos_tag to word net compatible format
# For this, we will use the function get_wordnet_pos -- you have to tag your␣
 ↪tokens first to utilize it

def get_wordnet_pos(treebank_tag): #no need to change this function - used to␣
 ↪tag tokens for context specification and then for lemmatization
    if treebank_tag.startswith('J'):
        return nltk.corpus.wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return nltk.corpus.wordnet.VERB
    elif treebank_tag.startswith('R'):
        return nltk.corpus.wordnet.ADV
    else:
        return nltk.corpus.wordnet.NOUN
```

```
def process(text, lemmatizer=nltk.stem.wordnet.WordNetLemmatizer()):
    """ Normalizes case and handles punctuation
    Inputs:
        text: str: raw text
        lemmatizer: an instance of a class implementing the lemmatize() method
                    (the default argument is of type nltk.stem.wordnet.
 ↪WordNetLemmatizer)
    Outputs:
        list(str): tokenized text
    """
    text = re.sub(r'http\S+', '', text)

    emoji_pattern = re.compile("["
        u"\U0001F600-\U0001F64F"  # emoticons
        u"\U0001F300-\U0001F5FF"  # symbols & pictographs
        u"\U0001F680-\U0001F6FF"  # transport & map symbols
        u"\U0001F1E0-\U0001F1FF"  # flags (iOS)
        u"\U00002702-\U000027B0"  # dingbats
        u"\U000024C2-\U0001F251"  # misc
                           "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r'', text)

    text = text.replace("'s", "")
    text = text.replace("'", "")

    text = re.sub(r'[' + string.punctuation + ']+', ' ', text).strip()

    text = text.lower()
    tokens = word_tokenize(text)

    tagged_tokens = nltk.pos_tag(tokens)
    lemma_tokens = [
        lemmatizer.lemmatize(word, get_wordnet_pos(pos))
        for (word, pos) in tagged_tokens
    ]

    return lemma_tokens
```

You can test the above function as follows. Try to make your test strings as exhaustive as possible. Some checks are:

[17]:
```
print(process("I'm doing well! How about you?"))
# ['im', 'do', 'well', 'how', 'about', 'you']

print(process("Education is the ability to listen to almost anything without␣
 ↪losing your temper or your self-confidence."))
```

```
# ['education', 'be', 'the', 'ability', 'to', 'listen', 'to', 'almost',␣
 ↪'anything', 'without', 'lose', 'your', 'temper', 'or', 'your', 'self',␣
 ↪'confidence']

print(process("been had done languages cities mice"))
# ['be', 'have', 'do', 'language', 'city', 'mice']

print(process("It's hilarious. Check it out http://t.co/dummyurl"))
# ['it', 'hilarious', 'check', 'it', 'out']

print(process("See it Sunday morning at 8:30a on RTV6 and our RTV6 app. http:
 ↪…"))
# ['see', 'it', 'sunday', 'morning', 'at', '8', '30a', 'on', 'rtv6', 'and',␣
 ↪'our', 'rtv6', 'app']
```

```
['im', 'do', 'well', 'how', 'about', 'you']
['education', 'be', 'the', 'ability', 'to', 'listen', 'to', 'almost',
'anything', 'without', 'lose', 'your', 'temper', 'or', 'your', 'self',
'confidence']
['be', 'have', 'do', 'language', 'city', 'mice']
['it', 'hilarious', 'check', 'it', 'out']
['see', 'it', 'sunday', 'morning', 'at', '8', '30a', 'on', 'rtv6', 'and', 'our',
'rtv6', 'app']
```

2.3 Q2 (9%):

You will now use the `process()` function we implemented to convert the pandas dataframe we just loaded from tweets_train.csv file. Your function should be able to handle any data frame which contains a column called `Content`. The data frame you return should replace every string in `Content` with the result of `process()` and retain all other columns as such. Do not change the order of rows/columns. Before writing `process_all()`, load the data into a DataFrame and look at its format:

[20]:
```
def process_all(df, lemmatizer=nltk.stem.wordnet.WordNetLemmatizer()):
    """ process all text in the dataframe using process() function.
    Inputs
        df: pd.DataFrame: dataframe containing a column 'Content' loaded from␣
 ↪the CSV file
        lemmatizer: an instance of a class implementing the lemmatize() method
                    (the default argument is of type nltk.stem.wordnet.
 ↪WordNetLemmatizer)
    Outputs
        pd.DataFrame: dataframe in which the values of Content column have been␣
 ↪changed from str to list(str),
                      the output from process() function. Other columns are␣
 ↪unaffected.
    """
```

```
    processed_df = df.copy()

    def safe_process(x):
        if isinstance(x, str):
            return process(x, lemmatizer=lemmatizer)
        return []

    processed_df["Content"] = processed_df["Content"].apply(safe_process)

    return processed_df
```

[21]:
```
# test your code
processed_tweets = process_all(train)
#print(processed_tweets["Content"])
print(processed_tweets.head())

#                                              Content          handle
# 0  [this, be, a, disgusting, and, heartbreaking, ...  Zohran Mamdani
# 1  [honor, to, join, richmond, hill, s, hindu, co...    Curtis Sliwa
# 2  [new, york, tomorrow, be, the, big, day, the, ...    Andrew Cuomo
# 3  [the, art, of, the, deal, break, news, mayor, ...  Zohran Mamdani
# 4  [october, 7th, be, a, dark, day, and, the, mem...    Curtis Sliwa
```

```
                                             Content          handle
0  [this, be, a, disgusting, and, heartbreaking, …  Zohran Mamdani
1  [honor, to, join, richmond, hill, ', s, hindu,…    Curtis Sliwa
2  [new, york, tomorrow, be, the, big, day, the, …    Andrew Cuomo
3  [the, art, of, the, deal, break, news, mayor, …  Zohran Mamdani
4  [october, 7th, be, a, dark, day, and, the, mem…    Curtis Sliwa
```

2.4 B. Feature Construction [25%]

The next step is to derive feature vectors from the tokenized tweets. In this section, you will be constructing a bag-of-words TF-IDF feature vector. But before that, as you may have guessed, the number of possible words is prohibitively large and not all of them may be useful for our classification task. We need to determine which words to retain, and which to omit. A common heuristic is to construct a frequency distribution of words in the corpus and prune out the head and tail of the distribution. The intuition of the above operation is as follows. Very common words (i.e. stopwords) add almost no information regarding similarity of two pieces of text. Similarly with very rare words. NLTK has a list of in-built stop words which is a good substitute for head of the distribution. We will consider a word rare if it occurs only in a single document (row) in whole of `tweets_train.csv`.

2.5 Q3 (15%):

Construct a sparse matrix of features for each tweet with the help of `sklearn.feature_extraction.text.TfidfVectorizer` (documentation here). You need to

pass a parameter `min_df=2` to filter out the words occuring only in one document in the whole training set. Remember to ignore the stop words as well. You must leave other optional parameters (e.g., `vocab`, `norm`, etc) at their default values. But you may need to use parameters like `lowercase` and `tokenizer` to handle `processed_tweets` that is a `list` of tokens (not raw text).

[31]:
```
def identity(x):
    return x
def create_features(processed_tweets, stop_words):
    """ creates the feature matrix using the processed tweet text
    Inputs:
        processed_tweets: pd.DataFrame: processed tweets read from train/test␣
 ↪csv file, containing the column 'Content'
        stop_words: list(str): stop_words by nltk stopwords (after processing)
    Outputs:
        sklearn.feature_extraction.text.TfidfVectorizer: the TfidfVectorizer␣
 ↪object used
            we need this to tranform test tweets in the same way as train tweets
        scipy.sparse.csr.csr_matrix: sparse bag-of-words TF-IDF feature matrix
    """
    vectorizer = TfidfVectorizer(
        tokenizer=identity,
        preprocessor=identity,
        token_pattern=None,
        lowercase=False,
        stop_words=stop_words,
        min_df=2
    )

    X = vectorizer.fit_transform(processed_tweets)

    return vectorizer, X
```

[32]:
```
# execute this code
# It is recommended to process stopwords according to our data cleaning rules
processed_stopwords = list(np.concatenate([process(word) for word in␣
 ↪stopwords]))
(tfidf, X) = create_features(processed_tweets["Content"],processed_stopwords)
# Ignore warning
print(tfidf)
print(X.shape)
# Output (should be similar):
# TfidfVectorizer(lowercase=False, min_df=2,
#                 stop_words=[np.str_('a'), np.str_('about'), np.str_('above'),
#                             np.str_('after'), np.str_('again'),
#                             np.str_('against'), np.str_('ain'), np.
 ↪str_('all'),
#                             np.str_('be'), np.str_('an'), np.str_('and'),
```

```
#                                np.str_('any'), np.str_('be'), np.str_('aren'),
#                                np.str_('arent'), np.str_('a'), np.str_('at'),
#                                np.str_('be'), np.str_('because'), np.str_('be'),
#                                np.str_('before'), np.str_('be'), np.
 ↪str_('below'),
#                                np.str_('between'), np.str_('both'), np.
 ↪str_('but'),
#                                np.str_('by'), np.str_('can'), np.str_('couldn'),
#                                np.str_('couldnt'), ...],
#                  tokenizer=<function identity at 0x14990ad40>)
# (913, 1585)
```

```
TfidfVectorizer(lowercase=False, min_df=2,
                preprocessor=<function identity at 0x000001981EAFFA60>,
                stop_words=[np.str_('a'), np.str_('about'), np.str_('above'),
                            np.str_('after'), np.str_('again'),
                            np.str_('against'), np.str_('ain'), np.str_('all'),
                            np.str_('be'), np.str_('an'), np.str_('and'),
                            np.str_('any'), np.str_('be'), np.str_('aren'),
                            np.str_('arent'), np.str_('a'), np.str_('at'),
                            np.str_('be'), np.str_('because'), np.str_('be'),
                            np.str_('before'), np.str_('be'), np.str_('below'),
                            np.str_('between'), np.str_('both'), np.str_('but'),
                            np.str_('by'), np.str_('can'), np.str_('couldn'),
                            np.str_('couldnt'), …],
                token_pattern=None,
                tokenizer=<function identity at 0x000001981EAFFA60>)
(913, 1596)
```

2.6 Q4 (10%):

Also for each tweet, assign a class label (0 or 1) using its `handle`. Use 0 for `Zohran Mamdani`, 1 for `Curtis Sliwa`, and 2 for `Andrew Cuomo`.

[33]:
```
def create_labels(processed_tweets):
    """ creates the class labels from handle
    Inputs:
        processed_tweets: pd.DataFrame: tweets read from train file, containing␣
 ↪the column 'handle'
    Outputs:
        numpy.ndarray(int): dense binary numpy array of class labels
    """

    label_map = {
        "Zohran Mamdani": 0,
        "Curtis Sliwa": 1,
        "Andrew Cuomo": 2
    }
```

```
    y = processed_tweets["handle"].map(label_map)

    return y.to_numpy(dtype=int)
```

[34]:
```
# execute this code
y = create_labels(processed_tweets)
print(y)
#[0 1 2 0 1 0 2 0 0 2 0 2 ...
```

```
[0 1 2 0 1 0 2 0 0 2 0 2 1 2 1 0 0 1 0 2 2 0 1 0 2 0 1 2 1 0 0 1 1 1 1 0 0
 1 1 2 1 1 1 1 0 0 0 0 2 0 0 1 1 0 2 1 1 0 2 2 0 2 0 1 1 1 1 1 1 0 0 0 1 1
 0 0 0 1 0 1 1 2 0 1 0 0 0 0 1 1 1 1 0 1 1 2 1 1 2 2 1 0 1 0 1 2 0 1 0 0 2
 0 2 1 1 0 2 2 0 0 2 1 1 1 2 0 0 1 2 1 1 0 0 2 0 1 0 0 1 2 0 0 0 1 1 0 1 1
 2 0 0 0 0 1 1 2 2 1 1 2 0 1 0 2 2 1 1 2 1 1 2 2 0 1 1 1 1 0 1 0 1 1 0 1 2
 2 0 1 0 0 0 0 1 1 1 0 0 2 0 1 0 1 1 1 1 1 2 0 2 1 1 1 1 1 1 1 2 1 2 1 2 1
 2 2 2 1 2 1 0 0 1 1 1 0 0 2 0 1 0 2 1 2 1 1 0 0 0 0 1 2 2 1 1 0 1 0 1 1 1
 1 1 1 2 1 2 2 1 1 1 0 0 0 0 1 1 1 1 2 2 1 0 1 1 1 1 1 0 2 0 0 0 0 0 1 1 0
 1 1 1 1 1 0 0 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 2 1 2 1 0 1 0 1 2 1 0 1 1
 0 1 2 1 2 0 1 1 1 1 2 2 1 2 1 1 1 0 1 1 0 1 2 2 1 1 1 0 2 1 2 1 0 1 0 0 1
 2 1 1 0 1 0 2 2 1 1 2 2 0 0 1 0 1 0 1 0 1 2 0 1 0 2 1 0 0 0 0 1 1 1 0 0 0
 0 0 1 0 1 0 1 2 0 1 2 1 0 2 0 2 0 1 1 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 1
 0 1 0 0 1 2 1 1 2 2 0 0 2 1 1 1 0 1 0 0 0 1 0 2 0 1 0 0 0 1 1 1 0 0 2 1 1
 0 0 0 1 1 1 1 0 1 2 2 0 0 0 1 1 2 2 2 1 0 0 0 2 0 0 1 2 2 0 0 1 2 0 1 1 0
 1 0 1 0 2 1 0 0 0 1 0 2 0 0 0 0 1 1 2 1 2 1 0 0 2 2 1 1 1 0 0 2 1 0 0 1 1
 1 2 1 1 0 1 0 2 1 2 2 1 1 0 0 1 2 1 1 0 1 2 1 1 0 2 0 2 1 1 0 2 2 0 1 0 2
 0 0 1 0 0 1 1 2 1 0 1 2 0 1 0 2 0 0 0 0 1 2 2 1 2 2 0 1 2 1 1 1 0 1 1 0 2
 0 1 0 0 1 1 1 2 0 0 0 2 0 1 0 1 0 1 2 1 1 0 0 1 1 2 0 0 0 1 1 1 0 2 0 0 2
 0 0 0 1 1 1 0 1 0 2 2 1 0 0 0 0 1 0 0 0 1 1 0 0 2 2 0 1 1 0 0 1 0 1 1 1 0
 0 2 2 1 0 2 2 0 1 2 1 2 1 1 0 1 2 1 1 0 0 0 1 2 1 2 1 1 0 0 1 1 0 1 0 0 0
 0 1 0 1 2 0 1 1 0 0 1 1 1 1 1 1 2 1 2 0 0 2 0 0 1 1 0 0 0 1 2 0 1 0 0 0 0
 0 2 0 1 0 2 1 0 0 0 0 1 0 0 0 0 0 0 2 0 0 2 1 2 2 1 1 0 0 2 0 2 1 2 1 1 0 0
 1 0 0 2 0 1 1 1 1 1 1 2 1 1 0 0 1 1 2 1 0 2 0 1 0 1 1 1 2 0 2 0 1 0 1 0 1
 1 1 0 1 0 1 0 2 0 1 0 0 1 1 1 0 2 1 1 1 1 0 0 2 0 0 0 1 1 1 0 0 1 1 0 0 0
 1 2 2 1 0 0 1 0 0 1 1 0 0 1 0 1 1 0 1 1 1 0 0 0 1]
```

2.7 C. Classification [60%]

And finally, we are ready to put things together and learn a model for the classification of tweets. The classifier you will be using is `sklearn.linear_model.LogisticRegression` (Logistic Regression).

At the heart of is the concept of Regularization or Penalty, which determines how large/small the parmaeters will be and also zero out parameters if there are dependence in the features matrix. `sklearn`'s Logistic provides four regularization functions: `none`, `l2`, `l1`, `elasticnet` (details here and here). Only some solvers will be able to handle `l1` and `elasticnet`, so you will have to choose one for this homework yourself (please read documentation!).

Through the various functions you implement in this part, you will be able to learn a classifier,

score a classifier based on how well it performs, use it for prediction tasks and compare it to a baseline.

Specifically, you will carry out the following tasks (Q5-9) in order:

1. Implement and evaluate a simple baseline classifier MajorityLabelClassifier.
2. Implement the `learn_classifier()` function assuming `penalty` is always one of {`none`, `l2`, `l1`, `elasticnet`}.
3. Implement the `evaluate_classifier()` function which scores a classifier based on accuracy of a given dataset.
4. Implement `best_model_selection()` to perform cross-validation by calling `learn_classifier()` and `evaluate_classifier()` for different folds and determine which of the four penalties performs the best.
5. Go back to `learn_classifier()` and fill in the best penalty.

2.8 Q5 (10%):

To determine whether your classifier is performing well, you need to compare it to a baseline classifier. A baseline is generally a simple or trivial classifier and your classifier should beat the baseline in terms of a performance measure such as accuracy. Implement a classifier called `MajorityLabelClassifier` that always predicts the class equal to **mode** of the labels (i.e., the most frequent label) in training data. Part of the code is done for you. Implement the `fit` and `predict` methods. Initialize your classifier appropriately.

[61]:
```
# Skeleton of MajorityLabelClassifier is consistent with other sklearn␣
 ↪classifiers
from collections import Counter

class MajorityLabelClassifier():
    """
    A classifier that predicts the mode of training labels
    """
    def __init__(self):
        """
        Initialize your parameter here
        """
        self.majority_label = None

    def fit(self, X, y):
        """
        Implement fit by taking training data X and their labels y and finding␣
 ↪the mode of y
        i.e. store your learned parameter
        """
        counts = Counter(y)
        self.majority_label = counts.most_common(1)[0][0]
        return self

    def predict(self, X):
```

```
        """
        Implement to give the mode of training labels as a prediction for each␣
 ↪data instance in X
        return labels
        """
        num_samples = X.shape[0]
        return np.full(num_samples, self.majority_label, dtype=int)

# Report the accuracy of your classifier by comparing the predicted label of␣
 ↪each example to its true label
baselineClf = MajorityLabelClassifier()
# Use fit and predict methods to get predictions and compare it with the true␣
 ↪labels y
baselineClf.fit(X,y)
predictions = baselineClf.predict(X)

# Evaluate the classifier (e.g., using accuracy)
from sklearn.metrics import accuracy_score
train_accuracy = accuracy_score(y, predictions)
print(train_accuracy)
# print(training accuracy) should give 0.41621029572836804 #basically nreps/
 ↪total = 380/913
```

```
0.41621029572836804
```

2.9 Q6 (10%):

Implement the `learn_classifier()` function assuming `penalty` is always one of {`none`, `l1`, `l2`, `elasticnet`}. Stick to default values for any other optional parameters.

[52]:
```
def learn_classifier(X_train, y_train, penalty):
    """ learns a classifier from the input features and labels using the␣
 ↪penalty function supplied
    Inputs:
        X_train: scipy.sparse.csr.csr_matrix: sparse matrix of features, output␣
 ↪of create_features()
        y_train: numpy.ndarray(int): dense binary vector of class labels,␣
 ↪output of create_labels()
        penalty: str: penalty function to be used with classifier.␣
 ↪[none|l2|l1|elasticnet]
    Outputs:
        sklearn.linear_model.LogisticRegression: classifier learnt from data
    """

    if penalty == "none":
        classifier = LogisticRegression(
            penalty=None,
```

```
            solver="lbfgs",
            max_iter=1000
        )

    elif penalty == "l2":
        classifier = LogisticRegression(
            penalty="l2",
            solver="lbfgs",
            max_iter=1000
        )

    elif penalty == "l1":
        classifier = LogisticRegression(
            penalty="l1",
            solver="liblinear",
            max_iter=1000
        )

    elif penalty == "elasticnet":
        classifier = LogisticRegression(
            penalty="elasticnet",
            solver="saga",
            l1_ratio=0.5,
            max_iter=1000
        )

    else:
        raise ValueError(f"Invalid penalty type: {penalty}")

    classifier.fit(X_train, y_train)
    return classifier
```

[53]:
```
# execute code
classifier = learn_classifier(X, y, 'l2')
```

2.10 Q7 (10%):

Now that we know how to learn a classifier, the next step is to evaluate it, ie., characterize how good its classification performance is. This step is necessary to select the best model among a given set of models, or even tune hyperparameters for a given model.

There are two questions that should now come to your mind: 1. **What data to use? - Validation Data**: The data used to evaluate a classifier is called **validation data** (or hold-out data), and it is usually different from the data used for training. The model or hyperparameter with the best performance in the held out data is chosen. This approach is relatively fast and simple but vulnerable to biases found in validation set. - **Cross-validation**: This approach divides the dataset in k groups (so, called k-fold cross-validation). One of group is used as test set for evaluation and other groups as training set. The model or hyperparameter with the best average performance

across all k folds is chosen. For this question you will perform 4-fold cross validation to determine the best penalty. We will keep all other hyperparameters default for now. This approach provides robustness toward biasness in validation set. However, it takes more time.

2. **And what metric?** There are several evaluation measures available in the literature (e.g., accuracy, precision, recall, F-1,etc) and different fields have different preferences for specific metrics due to different goals. We will go with accuracy. According to wiki, **accuracy** of a classifier measures the fraction of all data points that are correctly classified by it; it is the ratio of the number of correct classifications to the total number of (correct and incorrect) classifications. `sklearn.metrics` provides a number of performance metrics.

Now, implement the following function.

[48]:
```
def evaluate_classifier(classifier, X_validation, y_validation):
    """ evaluates a classifier based on a supplied validation data
    Inputs:
        classifier: sklearn.linear_model.LogisticRegression: classifer to␣
 ↪evaluate
        X_validation: scipy.sparse.csr.csr_matrix: sparse matrix of features
        y_validation: numpy.ndarray(int): dense binary vector of class labels
    Outputs:
        double: accuracy of classifier on the validation data
    """
    y_prediction = classifier.predict(X_validation)

    return accuracy_score(y_validation, y_prediction)
```

[47]:
```
# test your code by evaluating the accuracy on the training data
accuracy = evaluate_classifier(classifier, X, y)
print(accuracy)
# should give around 0.8817086527929902
```

```
0.8806133625410734
```

2.11 Q8 (10%):

Now it is time to decide which penalty works best by using the cross-validation technique. Write code to split the training data into 4-folds (75% training and 25% validation) by shuffling randomly. For each penalty, record the average accuracy for all folds and determine the best classifier. Since our dataset is balanced (both classes are in almost equal propertion), `sklearn.model_selection.KFold` doc can be used for cross-validation.

[50]:
```
kf = sklearn.model_selection.KFold(n_splits=4, random_state=1, shuffle=True)
kf
```

[50]:
```
KFold(n_splits=4, random_state=1, shuffle=True)
```

Then use the following code to determine which classifier is the best.

[54]:
```
def best_model_selection(kf, X, y):
    """
    Select the penalty giving best results using k-fold cross-validation.
    Other parameters should be left default.
    Input:
    kf (sklearn.model_selection.KFold): kf object defined above
    X (scipy.sparse.csr.csr_matrix): training data
    y (array(int)): training labels
    Return:
    best_penalty (string)
    """
    penalties = ['none', 'l2', 'l1', 'elasticnet']
    best_penalty = None
    best_avg_acc = -1.0

    for penalty in ['none', 'l2', 'l1', 'elasticnet']:
        fold_accuracies = []

        for train_index, val_index in kf.split(X):
            X_train, X_val = X[train_index], X[val_index]
            y_train, y_val = y[train_index], y[val_index]

            clf = learn_classifier(X_train, y_train, penalty)

            acc = evaluate_classifier(clf, X_val, y_val)
            fold_accuracies.append(acc)

        avg_acc = np.mean(fold_accuracies)

        if avg_acc > best_avg_acc:
            best_avg_acc = avg_acc
            best_penalty = penalty
        # Use the documentation of KFold cross-validation to split ..
        # training data and test data from create_features() and create_labels()
        # call learn_classifer() using training split of kth fold
        # evaluate on the test split of kth fold
        # record avg accuracies and determine best model (penalty)
    #return best penalty as string
    return best_penalty

#Test your code
best_penalty = best_model_selection(kf, X, y)
best_penalty
```

```
C:\Users\Dylan\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n
2kfra8p0\LocalCache\local-packages\Python313\site-
packages\sklearn\linear_model\_logistic.py:1296: FutureWarning: Using the
'liblinear' solver for multiclass classification is deprecated. An error will be
```

```
raised in 1.8. Either use another solver which supports the multinomial loss or
wrap the estimator in a OneVsRestClassifier to keep applying a one-versus-rest
scheme.
  warnings.warn(
C:\Users\Dylan\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n
2kfra8p0\LocalCache\local-packages\Python313\site-
packages\sklearn\linear_model\_logistic.py:1296: FutureWarning: Using the
'liblinear' solver for multiclass classification is deprecated. An error will be
raised in 1.8. Either use another solver which supports the multinomial loss or
wrap the estimator in a OneVsRestClassifier to keep applying a one-versus-rest
scheme.
  warnings.warn(
C:\Users\Dylan\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n
2kfra8p0\LocalCache\local-packages\Python313\site-
packages\sklearn\linear_model\_logistic.py:1296: FutureWarning: Using the
'liblinear' solver for multiclass classification is deprecated. An error will be
raised in 1.8. Either use another solver which supports the multinomial loss or
wrap the estimator in a OneVsRestClassifier to keep applying a one-versus-rest
scheme.
  warnings.warn(
C:\Users\Dylan\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n
2kfra8p0\LocalCache\local-packages\Python313\site-
packages\sklearn\linear_model\_logistic.py:1296: FutureWarning: Using the
'liblinear' solver for multiclass classification is deprecated. An error will be
raised in 1.8. Either use another solver which supports the multinomial loss or
wrap the estimator in a OneVsRestClassifier to keep applying a one-versus-rest
scheme.
  warnings.warn(
```

[54]: '12'

2.12 Q9 (10%)

We're almost done! It's time to write a nice little wrapper function that will use our model to classify unlabeled tweets from tweets_test.csv file.

[59]:
```
def classify_tweets(tfidf, classifier, unlabeled_tweets):
    """ predicts class labels for raw tweet text
    Inputs:
        tfidf: sklearn.feature_extraction.text.TfidfVectorizer: the␣
↪TfidfVectorizer object used on training data
        classifier: sklearn.linear_model.LogisticRegression: classifier learned
        unlabeled_tweets: pd.DataFrame: tweets read from tweets_test.csv
    Outputs:
        numpy.ndarray(int): dense binary vector of class labels for unlabeled␣
↪tweets
    """
    processed_unlabeled = process_all(unlabeled_tweets)
```

```
    X_unlabeled = tfidf.transform(processed_unlabeled["Content"])
    y_pred = classifier.predict(X_unlabeled)

    return y_pred
```

[62]:
```
# Fill in best classifier in your function and re-trian your classifier using␣
 ↪all training data
# Get predictions for unlabelled test data
classifier = learn_classifier(X, y, best_penalty)
unlabeled_tweets = pd.read_csv("test.csv", na_filter=False)
y_pred = classify_tweets(tfidf, classifier, unlabeled_tweets)
y_pred
```

[62]:
```
array([0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0,
       1, 2, 1, 0, 1, 1, 0, 0, 0, 2, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 2, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 2, 1, 0, 1,
       0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 1, 1, 0, 1, 0,
       2, 0, 1, 1, 1, 1, 0, 2, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0,
       1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 2, 0, 2, 0, 1,
       0, 1, 0, 1, 0, 1, 1])
```

Did your Logistic Regression classifier perform better than the baseline (while evaluating with training data)? Explain in 1-2 sentences how you reached this conclusion.

The Logistic Regression classifier did perform better than the baseline. The baseline accuracy was 0.41621029572836804 and the Logistic Regression accuracy was 0.8806133625410734

2.13 Notebook Cells Executions (10%)

The remaining 10% of points will be awarded for running and displaying the output of *all* cells in the notebook, so be sure to double check your executions.

[ ]:
```

```